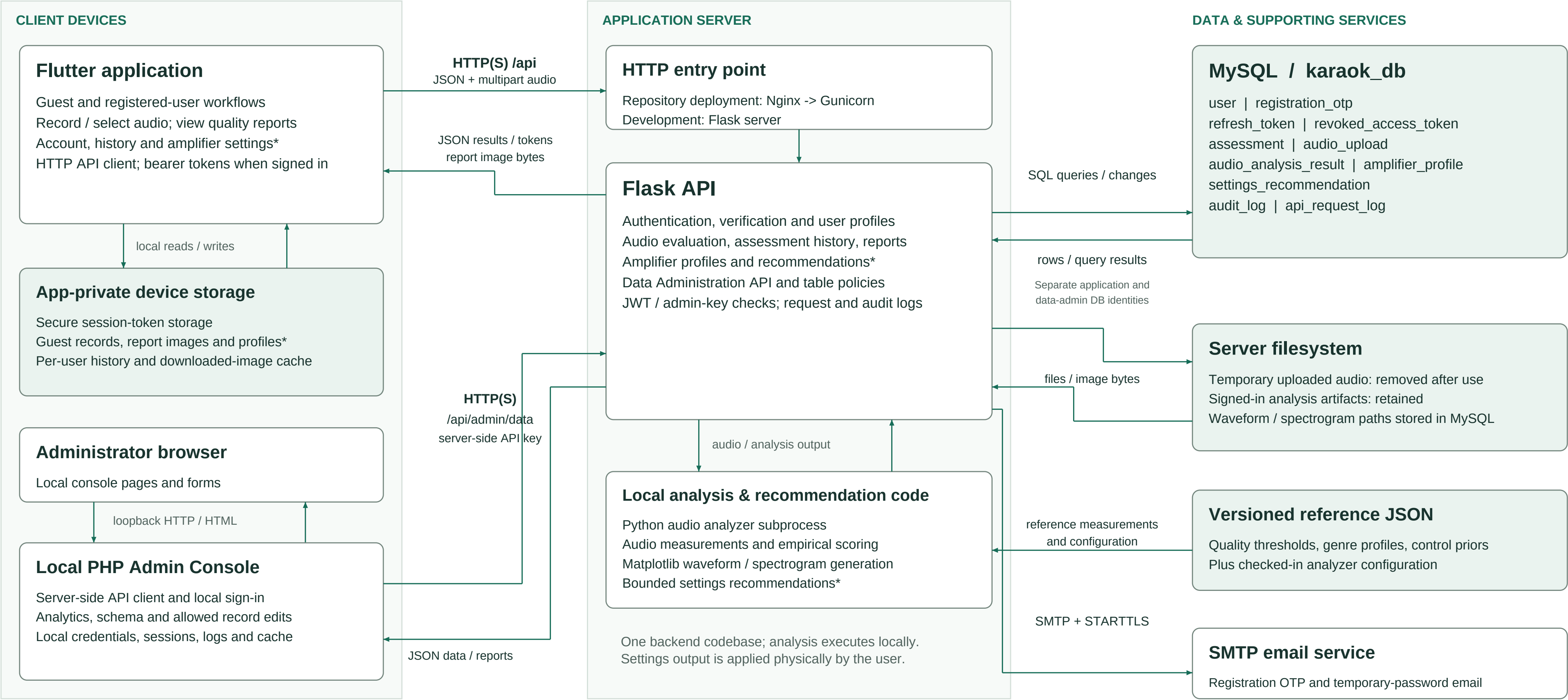


Client-server architecture

Flutter and the local PHP Admin Console use the Flask API; the API owns access to MySQL and analysis files.



* Implemented behind a settings feature flag (default: off). Deployment boxes describe checked-in configuration; live deployment was not queried.

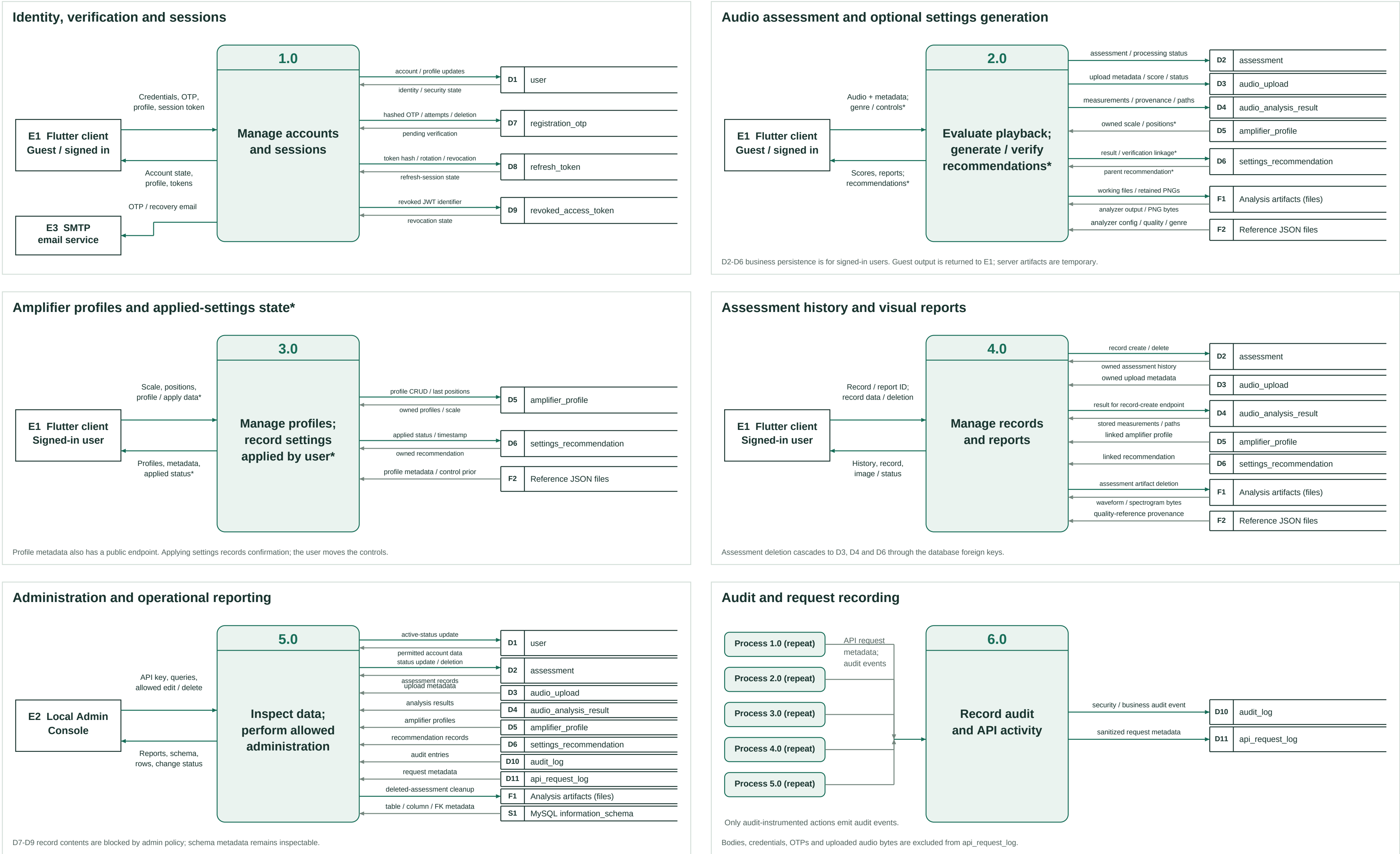
Guest assessment data and report images stay device-local; the API still analyzes guest audio and records request / audit metadata.

Source: current repository b74ceb8, database/schema.sql and implementation files. Detailed evidence: docs/diagrams/evidence.md.

Level 1 data-flow diagram

Scope: KaraOK server-side application. E1 = Flutter client; E2 = local Admin Console; E3 = SMTP email service.

Six numbered processes. Store IDs repeat to avoid crossing lines; a repeated ID always denotes the same store.



Notation: square = external entity; rounded rectangle = process; open-ended store = data store; arrows = named data flows.

D1-D11 exactly match database/schema.sql. F1-F2 are filesystem stores; S1 is MySQL system metadata. These are not additional ERD tables.

* Conditional settings feature (default: off). Shared authentication checks for protected requests reuse process 1.0. E1 owns device-local guest history.

Source: repository b74ceb8. Flow details and explicit limitations: docs/diagrams/evidence.md. This is a code-derived model, not a live-server inventory.